PyPortal

Table des matières

1 Analyse préliminaire 3
1.1 Introduction 3
1.2 Objectifs 3
1.2.1 Objectif général 3
1.2.2 Objectifs fonctionnels 3
1.2.3 Objectifs techniques 4
1.3 Planification initiale 5
1.3.1 Temps planifié initial 5
1.3.2 Tâches prévues initialement 5

2 Analyse / Conception 5
2.1 Méthode utilisée 5
2.1.1 Pivot technologique : de IceScrum à GitHub Project 6
2.1.2 Cycle de vie d'une tâche (Workflow) 6
2.2 Concept 6
2.3 Stratégie de test 7
2.4 Risques techniques 9
2.5 Planification 10
2.6 Dossier de conception 10

3 Réalisation 11
3.1 Dossier de réalisation 11
3.2 Description des tests effectués 11
3.3 Erreurs restantes 11
3.4 Liste des documents fournis 12

4 Conclusions 12

5 Titre à définir 12
5.1 Glossaire 12

6 Annexes 14
6.1 Résumé du rapport du TPI / version succincte de la documentation 14
6.2 Sources – Bibliographie 14
6.3 Journal de travail 14
6.4 Manuel d'Installation 14
6.5 Manuel d'Utilisation 14
6.6 Archives du projet 14

NOTE L'INTENTION DES UTILISATEURS DE CE CANEVAS:
Toutes les parties en italiques sont là pour aider à comprendre ce qu'il faut mettre dans cette partie du document. Elles n'ont donc aucune raison d'être dans le document final.
De plus, en fonction du type de projet, il est tout à fait possible que certains chapitres ou paragraphes n'aient aucun sens. Dans ce cas il est recommandé de les retirer du document pour éviter de l'alourdir inutilement.

1 Analyse préliminaire

1.1 Introduction

Le projet va consister à développer une application web en langage python qui permet de jouer à un jeu et d'avoir son meilleur résultat ainsi que son dernier résultat enregistré.

Ce projet est réalisé pour mon projet pré-TPI me permettant d'exercer mes connaissances en Python, Pygame, Flask, html et css.

L'application a pour but :
- De créer un compte utilisateur et de s'y connecter
- De lancer un jeu et d'avoir son score enregistré
- De pouvoir avoir accès à son meilleur résultat ainsi qu'au classement entre utilisateur

En autre, l'application permettra de pouvoir jouer à un jeu ainsi que comparer son score avec d'autre joueur avec l'aide d'un classement.

1.2 Objectifs

Ce projet a pour objectif principal de concevoir et de réaliser une application interactive complète combinant un mini-jeu, un backend web, une base de données relationnelle et une interface utilisateur web. L'application doit démontrer la capacité à intégrer plusieurs technologies logicielles au sein d'une architecture cohérente, fonctionnelle et sécurisée.

1.2.1 Objectif général

Développer une application permettant à un utilisateur de s'inscrire, de se connecter, de jouer à un mini-jeu, puis de voir son score enregistré et comparé à ceux des autres joueurs via un classement en ligne, tout en respectant les bonnes pratiques de développement.

1.2.2 Objectifs fonctionnels

1.2.2.1 Gestion des utilisateurs

- Permettre l'inscription des utilisateurs avec un nom d'utilisateur, un mot de passe sécurisé (haché) et un email optionnel.
- Assurer l'authentification des utilisateurs via un système de connexion avec gestion de session.
- Offrir la possibilité de supprimer un compte utilisateur ainsi que l'ensemble des scores associés.

1.2.2.2 Mini-jeu interactif

- Développer un mini-jeu simple et rapide à l'aide de Pygame (jeu de réflexe, clic sur cibles ou esquive).
- Générer un score mesurable à la fin de chaque partie.
- Afficher le score final dans le jeu avant le retour à l'interface web.
-

1.2.2.3 Communication jeu / serveur

- Mettre en place une communication entre le jeu Pygame et le serveur Flask via une API REST.
- Envoyer automatiquement le score en fin de partie vers le backend.
- Vérifier l'authenticité de l'utilisateur avant l'enregistrement du score.
- Confirmer au joueur la bonne prise en compte de son score.

1.2.2.4 Gestion et stockage des données

- Concevoir une base de données relationnelle SQLite structurée autour de tables utilisateurs et scores.
- Garantir l'intégrité des données à l'aide de clés étrangères et de contraintes.
- Assurer la persistance des utilisateurs et des scores dans le temps.

1.2.2.5 Classement et affichage des résultats

- Afficher un classement des meilleurs scores (top 10) avec le nom du joueur et la date de la partie.
- Mettre à jour le classement de manière dynamique.
- Afficher les informations de l'utilisateur connecté sur l'interface web.

1.2.3 Objectifs techniques

- Intégrer efficacement les technologies suivantes : Flask, Pygame, SQLite, API REST, HTML/CSS.
- Mettre en place une API REST documentée et accessible via Swagger pour faciliter les tests et la maintenance.
- Respecter les bonnes pratiques de sécurité (hachage des mots de passe, vérification des requêtes, gestion des sessions).
- Développer une interface web simple, claire et responsive, utilisable sur ordinateur comme sur mobile.

1.3 Planification initiale

Dans ce paragraphe je décrirai comment le projet a initialement été découpé, par la suite nous verrons comment le projet a été fait et quels points auraient été bénéficiaire d'améliorer

1.3.1 Temps planifié initial

Durée	Type
18h	Analyse
45h	Implémentation
9h	Test
18h	Documentation

1.3.2 Tâches prévues initialement

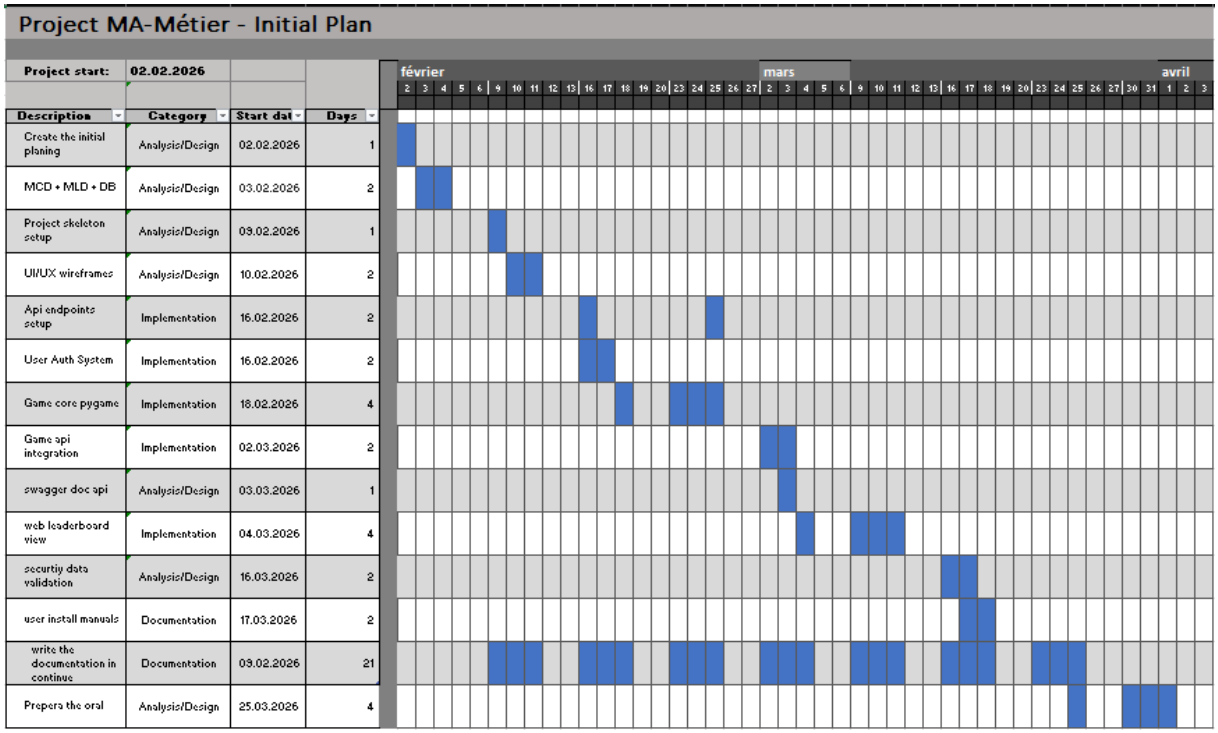

Figure 1 : planification initiale

2 Analyse / Conception

2.1 Méthode utilisée

La méthode utilisée pour ce projet est Agile. Le choix initial de la plateforme s'était porté sur IceScrum malheureusement due à des problème technique rencontré lors de son utilisation, la gestion a été migrée vers GitHub Project.

2.1.1 Pivot technologique : de IceScrum à GitHub Project

Bien que les 2 outils partagent une interface similaire basé sur le tableau Kanban (colonne To Do, In Progress, Done), ce changement a apporté des bénéfices significatifs en termes de flux de travail :

- **Centralisation :** L'intégration de GitHub Project au sein du dépôt de code permet de lier directement les storys aux commit et pull request
- **Traçabilité améliorée :** Chaque évolution du code est maintenant historisée dans <sa story> facilitant ainsi le suivis de celle-ci par les développeurs et le Product Owner.

Lien vers le Git Project : PyPortal Project

2.1.1.1 Les étapes du tableau Kanban

J'ai découpé le processus en 6 colonne principales :

1. **Sandbox** : Essertion de toutes les idées pouvant être intéressante pour le projet
2. **Backlog** : Contient toutes les user story validé par le PO. C'est ici que les priorité vont être définit
3. **Ready** : Contient les user story prête à être lancée. Elles contiennent toutes leurs tâches ainsi que leurs tests avec leur valeur.
4. **In Progress** : Lorsque que le développeur comment à travailler sur une tâche, ils déplace la story dans cette colonne
5. **In Review** : Une fois le code écrit, la story passe en phase de vérification. C'est à ce moment la que les checklists de tests manuels sont complétés dans la story.
6. **Done** : La tâche atteint cette colonne après le succès des tests et la fusion (merge) de la branche sur main.

2.1.1.2 Définition of Done (DoD)

Pour ce projet une tâche est considérée comme 'Done' uniquement si le code est commenté, les tests Pytest sont au vert, et la documentation Swagger a été mise à jour.

2.1.2 Cycle de vie d'une tâche (Workflow)

Pour assurer une progression fluide et transparante, j'ai créer un workflowau seins de Github Project. Cela permet de suivre un cheminement précis et guarantissant que rien n'a été mis en production sans avoir été testé et validé.

2.2 Concept

- Technologie : Python 3.12 (Flask 3.1.2, Pygame 2.6.1), les maquettes se trouveront au point ajouter points
- Outil de versionning : Git 2.51.2, le lien se trouve au point ajouter point
- IDE : Pycharm 2025.2.4 Professionnel
- Base de donnée : SQLite, le MCD et MLD se trouvent au point ajouter point
- Navigateur de test : Microsoft Edge

Le concept complet avec toutes ses annexes:

Par exemple :

- *Multimédia: carte de site, maquettes papier, story board préliminaire, …*
- *Bases de données: interfaces graphiques, modèle conceptuel.*
- *Programmation: interfaces graphiques, maquettes, analyse fonctionnelle…*
- *…*

2.3 Stratégie de test

Afin de garantir la longévité et la fiabilité de l'application web, une double stratégie de test a été mise en place, combinant validation fonctionnelle manuelle et automatisation continue (CI)

A. Validation fonctionnelle par User Stories

La première approche repose sur la définition de critères d'acceptation détaillés pour chaque User Story situer dans l'outil de gestion de projet (GitHub Project). En moyenne une User Story possède 3 tests fonctionnels permettant ainsi au développeur et au Product Owner (PO), de valider l'avancer du projet avant de changer l'état de celle-ci en Done (terminé et validé). Ces tests sont écrits d'une manière spécifique : Given, When, Then. Grâce cette méthode, souvent utilisé dans le Behavior-Driven Development, cela nous permet de décrire le comportement attendu d'un système de manière claire, structurée et compréhensible. Cela permet à une équipe ayant plusieurs rôles différents au sein de celle-ci (développeur, testeurs, etc…) de mieux se comprendre.
Elle définit un scénario de test en 3 étapes : Le contexte initial (Given), l'action réalisé (When) et le résultat attendu (Then).

- **Avantage :** Cette méthode permet une conformité exacte entre le besoin métier et la fonctionnalité livrée.
- **Limite :** Cependant cette méthode est sensible à la régression. Toute modification ultérieure du code nécessite de refaire passer l'état de la story en <In Progress>, permettant ainsi de réitérer les tests manuels.

Exemple d'une User Story :

- [x] Login
  - Given : a password and a pseudo
  - When : I click on submit
  - Then : I can access to the main page
- [x] Register
  - Given : A pseudo and a password
  - When : I click on submit
  - Then : I am redirect to the login page and my account is created

Figure 2 - Exemple de validation fonctionnelle dans une User Story

Ces deux test permettent dans l'ordre la validation de la connexion et de l'inscription.

B. Automatisation et intégration Continue (CI)

Pour palier les limites des tests manuels, une pipeline d'intégration continue a été configurée via GitHub Actions. Cette stratégie permet un contrôle systématique et automatique du projet à chaque évènement (push ou pull request) sur les branches stratégique (main et develop).

Le workflow automatisé exécute les étapes suivantes :

1. **Checkout du code** : Récupération des sources pour l'éxécution
2. **Configuration de l'environnement** : Installation de python dans la version demandée pour le projet
3. **Gestion des dépendances** : Installer toutes les dépendances via le gestionnaire de paquet (pip)
4. **Analyse statique (Linting)** : Vérifier le code avec flake8 afin de déceller des eventuelles erreurs de syntaxe ou un code qui ne respecterait pas les standards de style (PEP 8)
5. **Validation de l'infrastructure de données** : Lancement du fichier d'initialisation de la base de données permettant ainsi la vérification du shéma SQL et des contrainte

6. **Test unitaire et fonctionnels** : Lancement des suites de tests avec pytest, permettant ainsi de vérifier que les fonctions testées sont toujours opérationnelles

Exemple d'un GitHub actions ayant eu lieu lors d'une demande de pull :

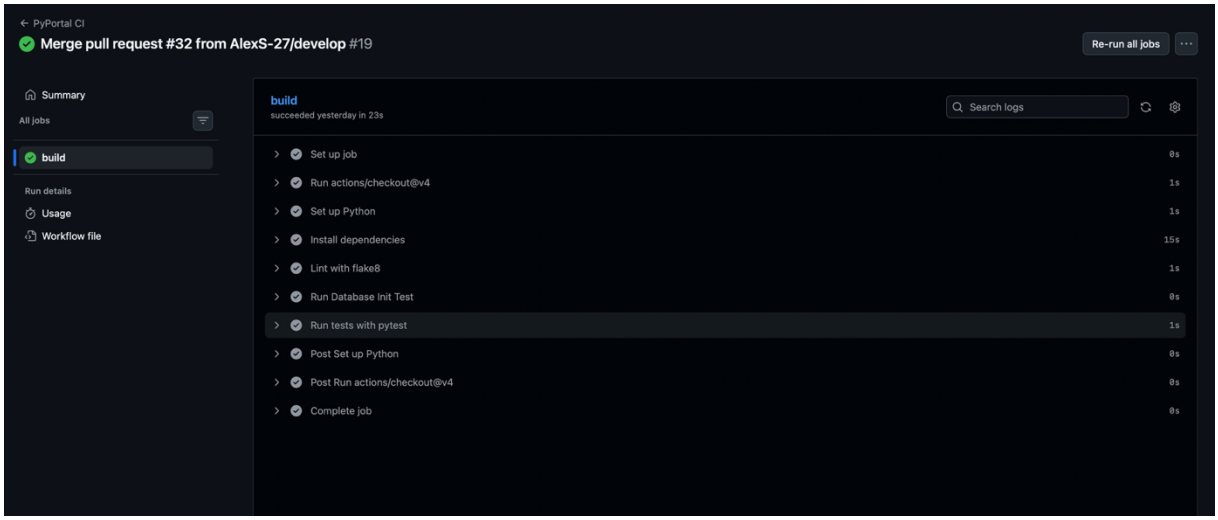

Figure 3 : Exemple Git Actions

C. *Bonus*

Bien que swagger ne soit pas un outil de test automatisé, il a été intégré pour servir de documentation et permettre une validation manuelle rapide des Endpoint pour le PO sans nécessiter d'outil tiers comme Insomnia

D. *Conclusion :*

En conclusion, l'assimilation de ces deux stratégies vise un objectif central : la couverture de code (Code Coverage).

Cette approche permet de maximiser la vérification des chemins logiques permettant ainsi une réduction significative sur les « zones d'ombre » du projet, minimisant ainsi le risque de bugs lattant. Elle offre une sérénité lors de l'ajout de nouvelle fonctionnalité, garantissant ainsi une évolution du code s'appuyant sur une base solide et fonctionnelle.

2.4 Risques techniques

A compléter ultérieurement lorsque la gestion des risques sera mieux analysé

- *risques techniques (complexité, manque de compétences, …).*

Décrire aussi quelles solutions ont été appliquées pour réduire les risques (priorités, formation, actions, …).

2.5 Planification

Voici la planification finale :

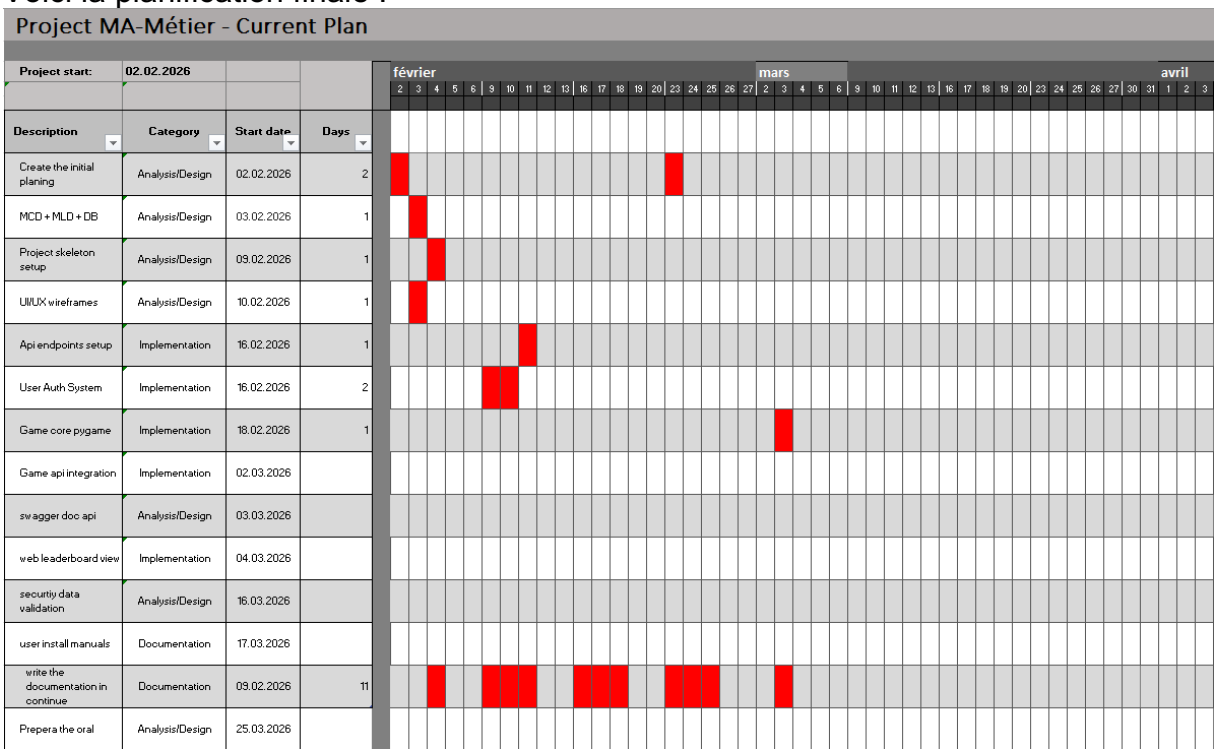

Figure 4 : Planification finale

Voir la planification initiale pour comparer

Révision de la planification initiale du projet :

- *planning indiquant les dates de début et de fin du projet ainsi que le découpage connu des diverses phases.*
- *partage des tâches en cas de travail à plusieurs.*

*Il s'agit en principe de la planification **définitive du projet**. Elle peut être ensuite affinée (découpage des tâches). Si les délais doivent être ensuite modifiés, le responsable de projet doit être avisé, et les raisons doivent être expliquées dans l'historique.*

2.6 Dossier de conception

Fournir tous les documents de conception:

- *le choix du matériel HW*
- *le choix des systèmes d'exploitation pour la réalisation et l'utilisation*
- *le choix des outils logiciels pour la réalisation et l'utilisation*

- *site web: réaliser les maquettes avec un logiciel, décrire toutes les animations sur papier, définir les mots-clés, choisir une formule d'hébergement, définir la méthode de mise à jour, …*
- *bases de données: décrire le modèle relationnel, le contenu détaillé des tables (caractéristiques de chaque champs) et les requêtes.*
- *programmation et scripts: organigramme, architecture du programme, découpage modulaire, entrées-sorties des modules, pseudo-code / structogramme…*

Le dossier de conception devrait permettre de sous-traiter la réalisation du projet !

3 Réalisation

3.1 Dossier de réalisation

Décrire la réalisation "physique" de votre projet

- *les répertoires où le logiciel est installé*
- *la liste de tous les fichiers et une rapide description de leur contenu (des noms qui parlent !)*
- *les versions des systèmes d'exploitation et des outils logiciels*
- *la description exacte du matériel*
- *le numéro de version de votre produit !*
- *programmation et scripts: librairies externes, dictionnaire des données, reconstruction du logiciel - cible à partir des sources.*

NOTE : Evitez d'inclure les listings des sources, à moins que vous ne désiriez en expliquer une partie vous paraissant importante. Dans ce cas n'incluez que cette partie…

3.2 Description des tests effectués

Pour chaque partie testée de votre projet, il faut décrire:

- *les conditions exactes de chaque test*
- *les preuves de test (papier ou fichier)*
- *tests sans preuve: fournir au moins une description*

3.3 Erreurs restantes

S'il reste encore des erreurs:

- *Description détaillée*
- *Conséquences sur l'utilisation du produit*
- *Actions envisagées ou possibles*

3.4 Liste des documents fournis

Lister les documents fournis au client avec votre produit, en indiquant les numéros de versions

- *le rapport de projet*
- *le manuel d'Installation (en annexe)*
- *le manuel d'Utilisation avec des exemples graphiques (en annexe)*
- *autres…*

4 Conclusions

Développez en tous cas les points suivants:

- *Objectifs atteints / non-atteints*
- *Points positifs / négatifs*
- *Difficultés particulières*
- *Suites possibles pour le projet (évolutions & améliorations)*

5 Titre à définir

5.1 Glossaire

Terme	Définition
API rest	
Appliation web	
Authencité	
Backend Web	
Base de donnée	
Base de données relationnelle	
Behavior-Driven Development	
Branche	
Chef de projet	
CI	
Code coverage	
Dépendance	
Etat de la story	
Flake8	
Flask	

Git Project	
GitActions	
Html/CSS	
IceScrum	
MCD	
MLD	
Mot de passe haché	
PEP 8	
Pipeline	
PO	
Pull	
Push	
Pycharm	
Pygame	
Pytest	
Python	
Shéma SQL	
SQLite	
User Story	
Versionning	
Workflow	
Endpoints	
Insomnia	
Kanban	
merge	

Table des figures :

FIGURE 1 : PLANIFICATION INITIALE 5
FIGURE 2 : EXEMPLE GIT ACTIONS 9
FIGURE 3 : PLANIFICATION FINALE 10

6 Annexes

6.1 Résumé du rapport du TPI / version succincte de la documentation

6.2 Sources – Bibliographie

Liste des livres utilisés (Titre, auteur, date), des sites Internet (URL) consultés, des articles (Revue, date, titre, auteur)… Et de toutes les aides externes (noms)

6.3 Journal de travail

Date	Durée	Activité	Remarques

6.4 Manuel d'Installation

6.5 Manuel d'Utilisation

6.6 Archives du projet

Media, … dans une fourre en plastique